FR

Fee Reconciliation Desk

FEE PAYMENT MATCHING AND RECONCILIATION

INSTALLATION AND ADMINISTRATION GUIDE

Installation and Administration Guide

Installing, configuring, hosting, securing and managing users

Document ID FRD-IG-02 · **Version** 1.0 · **Date** 27-Oct-2026

Prepared by Yasin Jariwala

Audience Administrators and IT or infrastructure teams

Document control

Document	Installation and Administration Guide
Document ID	FRD-IG-02
Version	1.0
Date	27-Oct-2026
Prepared by	Yasin Jariwala
Product	Fee Reconciliation Desk v1.0
Audience	Administrators and IT or infrastructure teams
Classification	Public
Source	github.com/yasin679/FeeReconciliationApp · docs/installation-and-administration.md

Version history

Version	Date	Author	Changes
1.0	27-Oct-2026	Yasin Jariwala	First release

Related documents

ID	Document
FRD-UG-01	User Guide
FRD-QS-03	Quick Start
FRD-FAQ-04	FAQ and Troubleshooting
FRD-TD-05	Technical Design
FRD-UAT-06	UAT Test Cases
FRD-RN-07	Release Notes v1.0

Contents

1 About this guide 3
1.1 What is in the repository 3
2 Requirements 4
2.1 Machine 4
2.2 Data you will need for go-live 4
3 Installation 5
3.1 Windows 5
3.2 Linux and macOS 5
3.3 The admin password 5
4 Configuration 6
4.1 Application settings 6
4.2 Bank statement format 6
5 Administration in the application 7
5.1 Policy controls 7
5.2 Users and roles 7
5.3 Demo accounts 7
6 Going live 9
7 Hosting for a team 10
8 Security, backup and logs 11
9 Upgrading 12
10 Running the automated tests 13
11 Go-live checklist 14

1 About this guide

This guide is for administrators who install, configure, host and support Fee Reconciliation Desk. Day-to-day users should read the User Guide (FRD-UG-01).

Fee Reconciliation Desk is a C# web application built on ASP.NET Core 8. It needs no database server and no third-party packages, and keeps its data in one file on the machine it runs on.

1.1 What is in the repository

Folder or file	Contents
`app/src/FeeReconDesk`	The C# source: `Domain` (rules engine, data model, sample data, self-tests), `Api` (endpoints, snapshots, CSV import and export), `Infra` (data store, passwords, audit chain, first start) and `wwwroot` (the screens)
`app/start.bat`	Starts the app on Windows and asks for the admin password
`app/build.bat`	Builds the app into `app/app` and runs the system check
`app/sample-bank-statement.csv`	An example bank statement in the accepted format
`prototype/`	Two single-file HTML previews that need no installation
`docs/`	These guides, as Markdown, with the PDFs in `docs/pdf`
`tests/`	Browser end-to-end, API security, concurrency and fuzz tests

Ready-built or from source

The release package `Fee-Reconciliation-Desk.zip` already contains the built `app` folder, so only the runtime is needed. The Git repository holds source only, so run `build.bat` once (which needs the SDK) before `start.bat`.

2 Requirements

2.1 Machine

Item	Requirement
Operating system	Windows 10 or 11. Linux and macOS also work.
Runtime	ASP.NET Core 8 runtime: `winget install Microsoft.DotNet.AspNetCore.8`
To build from source	.NET 8 SDK: `winget install Microsoft.DotNet.SDK.8`
Memory and disk	Any machine that runs Windows 10. The data is one JSON file.
Browser	A current Edge, Chrome or Firefox
Network	None to build or run. It listens on `http://localhost:5090` by default.

2.2 Data you will need for go-live

Item	Source
Staff list with roles	Department heads. One account per person.
At least two Finance approvers	Finance. Approvals and corrections need two different people.
Student list (ID, name, registered contact, guardian)	Student-management system
Daily bank statement export (CSV)	College bank account portal
Fee policy decisions (component order, cash deposit window, evidence standard)	Finance Controller

3 Installation

3.1 Windows

1. Install the runtime (or the SDK if you are building from source).
2. Copy or unzip the **Fee Reconciliation Desk App** folder to the machine.
3. From source only: double-click `build.bat`. It publishes into `app` and runs the 27-scenario system check; every scenario should pass.
4. Double-click `start.bat`.
5. At **New admin password**, type a password and press Enter. It is hidden as you type. The console confirms *Admin password set from FRD_ADMIN_PASSWORD*.
6. With sample data on, the console also prints one shared demo password, in yellow, once. Copy it now.
7. The browser opens http://localhost:5090. Sign in as `admin`.
8. Keep the console window open while the app is in use. Closing it stops the app.

On later starts, press Enter at the prompt to keep the current password.

3.2 Linux and macOS

```
cd app/src/FeeReconDesk
dotnet publish -c Release -o ../../app
cd ../../app
FRD_ADMIN_PASSWORD='Choose-A-Str0ng-One' dotnet FeeReconDesk.dll
```

3.3 The admin password

Situation	What happens
A password is typed at the `start.bat` prompt (or `FRD_ADMIN_PASSWORD` is set)	The `admin` password is set to it, and `admin` is re-enabled and unlocked
The prompt is skipped on the very first start	A temporary admin password is printed once, in yellow
The password does not meet the rules	The console says *Admin password ignored* and the old password stays

Password rules: at least 10 characters with three of upper case, lower case, digits and symbols.

4 Configuration

4.1 Application settings

Settings live in `app/appsettings.json` and need a restart. Any setting can also be given as an environment variable of the same name.

Setting	Default	Meaning
`Urls`	`http://localhost:5090`	Address the app listens on. Keep it on localhost unless it sits behind HTTPS.
`DataPath`	`data`	Folder for the data file, its backup and the sign-in keys, relative to `app`
`SessionTimeoutMinutes`	30	Sign-out after inactivity (5–480)
`LoginAttemptsPerMinute`	30	Sign-in attempts allowed per network address per minute. Raise it if many staff share one office address.
`EnableDemoData`	true	Loads the sample data and demo accounts on first start. Set to false before the first real start.
`BehindProxy`	false	Set to true only behind a trusted reverse proxy that sends X-Forwarded headers
`AllowedHosts`	`localhost;127.0.0.1`	Host names the app answers to. Add the server's name when hosting it for others.

4.2 Bank statement format

The first row holds column names. Matching ignores case, spaces and underscores, and common bank column names are accepted (`UTR`, `Narration`, `Remitter`, `Mobile`).

Column	Required	Notes
`TransactionId`	Yes	Bank UTR or reference. The duplicate check ignores case, spaces and punctuation.
`Amount`	Yes	Whole rupees; commas and ₹ are allowed. A negative amount is a reversal. Up to ₹10 crore.
`Date`	No	`2026-10-26`, `26-10-2026`, `26/10/2026` or `26-Oct-2026`
`PayerName`, `PayerContact`	No	Used only to suggest candidates
`Remark`	No	A student ID here (`S101`) matches automatically. `S101 15000 / S104 15000` becomes a split.
`ReversalOf`	No	The original `TransactionId`, for reversals
`Channel`	No	UPI, NEFT, IMPS and so on

One file holds up to 20,000 rows and 5 MB.

5 Administration in the application

5.1 Policy controls

Matching and allocation policies are changed under **Policy controls** (administrators only). They take effect for payments imported afterwards and are recorded in the audit log.

Policy	Default	Meaning
AM-1 Auto-match on a valid student ID	On	A verified bank line with exactly one valid student ID and no conflicting signal is matched without a person
AA-1 Fee-component allocation order	No policy	*Admission first, then tuition* allocates and posts matched payments automatically. Turn it on only with Finance's written approval.
Claims under review per student	5	Limits unconfirmed claims and open cases per student
Fail first posting attempt	Off	Sample data only: simulates a student-system timeout

5.2 Users and roles

Each person needs their own account so the audit log and the two-person checks know who did what.

Task	How
Add a user	Enter user name, display name and role (and the student ID for a student), then **Add user**. A temporary password is shown once; share it securely.
Change a role	Pick the new role in the user's row. Their open session ends immediately.
Disable a user	**Disable** in their row. They are signed out on their next request; their history stays in the audit log.
Reset a password	**Reset password** in their row. A new temporary password is shown once, and any lockout is cleared.
Change your own password	**Password** at the top right. You are signed out and sign in again.
Recover the admin account	Close the app, run `start.bat` and type a new password at the prompt

You cannot disable your own account or remove your own administrator access. Administrators cannot propose, approve, post or correct money; give a person a Finance role for that.

5.3 Demo accounts

With sample data on, these accounts share the password printed at first start:

User name	Role	Student
`meera`	Operations executive	—
`priya`	Counsellor	—

User name	Role	Student
`arjun`	Finance reviewer	—
`kavita`, `rahul`	Finance approvers	—
`ananya`, `rohan`, `zara`, `dev`	Students	S101, S102, S103, S104

6 Going live

1. Before the first start, set `"EnableDemoData": false` in `app/appsettings.json`. If the app has already run with sample data, close it and rename the `app/data` folder first; a new empty book is created on the next start.
2. Start the app, sign in as `admin`, and add one account per person.
3. Add students under **Add payments → Add student**, or load them from the student-management system.
4. Agree the policy decisions with the Finance Controller, especially whether AA-1 may be turned on.
5. Finance imports the bank statement each day.

7 Hosting for a team

For more than one person, run the app on a server rather than a laptop:

1. Host it behind HTTPS: IIS with the ASP.NET Core Hosting Bundle, Azure App Service, or a reverse proxy such as nginx.
2. Set `BehindProxy` to true and add the server's host name to `AllowedHosts`.
3. Restrict who can reach it, for example with the college network or VPN.
4. Run one instance only. The data file is not designed to be shared between instances.

8 Security, backup and logs

Where	What is stored
`app/data/fee-recon-desk.json`	Students, payments, allocations, ledger, receipts, cases, policy, users (password hashes only) and the audit log
`app/data/fee-recon-desk.json.bak`	The previous version of the data file, refreshed on every save
`app/data/keys/`	Keys that sign session cookies, encrypted with Windows DPAPI on Windows

Built-in protections

- Every action is checked on the server: the role, the two-person rule by user, and the money rules. The page only displays.
- Students receive only their own data. A claim never reveals whether a payment exists until the money is allocated to that student.
- Passwords are stored as salted PBKDF2-SHA256 hashes with 210,000 iterations. Accounts lock for 10 minutes after 5 failures, and sign-in is rate-limited per address.
- Session cookies are HttpOnly and SameSite=Strict. Every change needs an anti-forgery token. A strict Content Security Policy is applied, and no external web resources are loaded.
- The audit log is append-only and hash-chained; **Verify integrity** detects any edit.
- CSV exports neutralise spreadsheet formulas.

Backup: back up `app/data` daily, copying the data file and the `keys` folder together. To restore, stop the app, put the files back and start it. If the data file is ever damaged, rename it and rename the `.bak` file to take its place.

Logs: the console window shows start-up messages and warnings. Who did what is in the audit log, not the console.

9 Upgrading

1. Back up `app/data`.
2. Replace every file in `app` except the `data` folder and your `appsettings.json`.
3. Start the app and run **System check**.

10 Running the automated tests

Test	Command	Expected
Built-in system check	`dotnet app/FeeReconDesk.dll --selftest`	27 / 27 pass
Browser end-to-end	`node tests/e2e.js` (server running, Playwright installed)	14 / 14 pass
API security	`node tests/secapi.js`	0 findings open
Concurrency	`node tests/race.js`	1 proposal, 1 approval, 1 posting

11 Go-live checklist

Check	Done
`EnableDemoData` is false and no demo accounts exist	
Hosted behind HTTPS with `BehindProxy` and `AllowedHosts` set (if shared)	
Every person has their own account; at least two Finance approvers	
The admin password is strong and known to two people	
Policy decisions signed off by the Finance Controller	
Daily backup of `app/data` scheduled and one restore tested	
System check shows 27 / 27	
One day's bank statement imported in a trial run, and the control difference is ₹0	